บทที่ 9

ฟังก์ชันในภาษาซี

Function in C Language

ทำไมถึงใช้ฟังก์ชัน

หากต้องการเขียนโปรแกรมเพื่อหาค่าผลรวมจาก 1 ถึง 10 และจาก 20 ถึง 37

การเขียนคำสั่ง
โดยไม่เรียกใช้ฟังก์ชัน

ต้องเขียนคำสั่งเดิม 2 ครั้ง

```
int i, sum = 0;
for (i=1; i<=10; i++)
  sum +=i;
printf("Sum from 1 to 10 is %d", sum);
sum = 0;
for (i=20; i<=37; i++)
  sum +=i;
printf("Sum from 20 to 37 is %d", sum);
```

การเขียนคำสั่ง
โดยเรียกใช้ฟังก์ชัน

```
int sum(int i1, int i2){
  int i, sum = 0;
  for(i=i1, i<=i2; i++)
    sum += i;
return sum; }

int main(){
printf("Sum from 1 to 10 is %d\n",sum(1,10));
printf("Sum from 20 to 37 is %d",sum(20,37));
}
```

ฟังก์ชันคืออะไร

ในการเขียนโปรแกรมคอมพิวเตอร์บางครั้งมีชุดคำสั่งบางชุดที่ถูกเรียกใช้งานบ่อย หากต้องเขียนชุดคำสั่งนั้นบ่อยครั้ง จะทำให้โปรแกรมมีขนาดใหญ่ จึงควรนำเอาชุดคำสั่งเหล่านั้นมารวมเป็นฟังก์ชัน

- ฟังก์ชัน คือ กลุ่มของคำสั่งที่สามารถถูกเรียกใช้ได้เพื่อทำงานเฉพาะอย่าง
- ประโยชน์ที่สำคัญของฟังก์ชัน คือ ช่วยลดความซ้ำซ้อนของโปรแกรม อ่านเข้าใจ และแก้ไขได้ง่าย

ฟังก์ชัน (Function)

ฟังก์ชันในภาษาซีแบ่งออกเป็น 2 ประเภท คือ

- **ฟังก์ชันมาตรฐานของภาษาซี (Standard Library Functions)**
  - ฟังก์ชันที่ภาษาซีจัดเตรียมไว้ให้แล้ว เพียงแต่ต้องทราบก่อนว่าฟังก์ชันนั้นมีการประกาศไว้ในเฮดเดอร์ไฟล์ใด แล้วเพิ่มบรรทัดคำสั่ง **#include <ชื่อเฮดเดอร์ไฟล์>**
  - ตัวอย่างเช่น #include<stdio.h> ← กำหนดไว้ที่หัวโปรแกรม
    ถ้าไม่มีจะไม่สามารถเรียกใช้ scanf() หรือ printf()
- **ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง (User-defined function)**
  - ฟังก์ชันที่นักพัฒนาโปรแกรมเป็นผู้เขียนขึ้นมาเอง **โดยกำหนดชื่อ รูปแบบการรับข้อมูลขาเข้า รูปแบบการประมวลผลข้อมูล และรูปแบบการคืนค่าข้อมูลขาออก**
    ซึ่งสามารถเรียกใช้ซ้ำได้หลายครั้งภายในโปรแกรม

main() ถือเป็นฟังก์ชันชนิดหนึ่งที่เป็นจุดเริ่มต้นทำงานของโปรแกรม

ฟังก์ชันมาตรฐาน (Standard Function)

Standard Function เป็นฟังก์ชันสำเร็จรูปที่มีมาให้พร้อมกับ Compiler โดยเก็บอยู่ในไฟล์ *.h ซึ่งมีรูปแบบดังนี้ #include <file.h>

Library Function ที่สำคัญ

- math.h ฟังก์ชันการคำนวณทางคณิตศาสตร์
- string.h ฟังก์ชันสำหรับข้อความ
- ctype.h ฟังก์ชันสำหรับอักขระ

ฟังก์ชันการคำนวณทางคณิตศาสตร์ ใน math.h

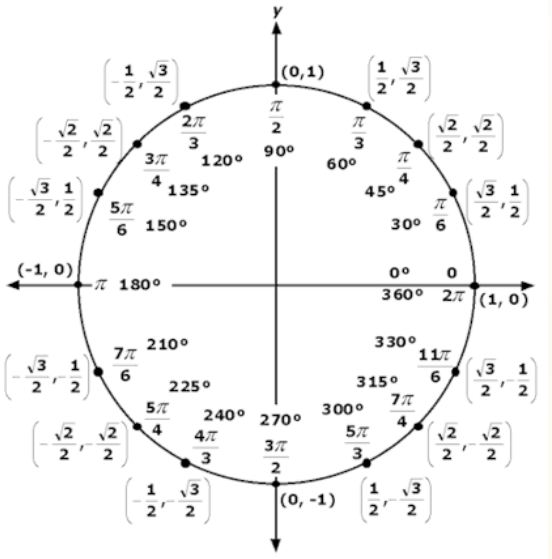
Function	Description
sin(x)	sine ของ x โดย x มีหน่วยเป็นเรเดียน
cos(x)	cosine ของ x โดย x มีหน่วยเป็นเรเดียน
tan(x)	tangent ของ x โดย x มีหน่วยเป็นเรเดียน
sqrt(x)	รากที่สองของ x โดย x>=0
pow(x,y)	x ยกกำลัง y
exp(x)	e ยกกำลัง x
log(x)	logarithm ฐาน e ของ x โดย x>=0
log10(x)	logarithm ฐาน 10 ของ x โดย x>=0
abs(x)	ค่าสัมบูรณ์ของ x โดย x เป็นเลขจำนวนเต็ม
fabs(x)	ค่าสัมบูรณ์ของ x โดย x เป็นเลขจำนวนจริง

$$\text{Radians} = \left(\frac{\pi}{180^{0}}\right) \times \text{degrees}$$

$$\text{Degrees} = \left(\frac{180^{0}}{\pi}\right) \times \text{radians}$$

ตัวอย่างการใช้ฟังก์ชัน math.h

```
1  | #include <stdio.h>
2  | #include <math.h>
3  | double x1 = 3.14159, x2 = 2.71828, y;
4  | int main(void){
5  |   y = sin(x1);
6  |   printf("sin(pi)    = %7.4f\n", y);
7  |   printf("cos(pi/4) = %7.4f\n", cos(x1/4));
8  |   printf("tan(pi/6) = %7.4f\n", tan(x1/6));
9  |   printf("sqrt(2)    = %7.4f\n", sqrt(2));
10 |   printf("2.5^3      = %7.4f\n", pow(2.5,3));
11 |   printf("loge(e)    = %7.4f\n", log(x2));
12 |   printf("log(100)   = %7.4f\n", log10(100));
13 |   printf("e^3        = %7.4f\n", exp(3));
14 |   printf("abs(-7)    = %7.4f\n", abs(-7));
15 |   printf("fabs(-7.0)= %7.4f\n", fabs(-7.0));
16 |   return(0);
17 | }
```



x = cos(θ) และ y = sin(θ)

```
sin(pi)    =  0.0000
cos(pi/4)  =  0.7071
tan(pi/6)  =  0.5774
sqrt(2)    =  1.4142
2.5^3      = 15.6250
loge(e)    =  1.0000
log(100)   =  2.0000
e^3        = 20.0855
abs(-7)    =  7.0000
fabs(-7.0)=  7.0000
```

ฟังก์ชันสำหรับข้อความ ใน string.h

Function	Description
strcpy(str1, str2)	คัดลอกข้อความจาก str2 ไปเก็บที่ str1
strcat(str1, str2)	ต่อข้อความใน str1 ด้วย str2
strncat(str1,str2,n)	ต่อข้อความใน str1 ด้วย str2 จำนวน n อักษร
strcmp(str1, str2)	เปรียบเทียบตัวอักษรในข้อความ (case sensitive) ถ้า str1 = str2 จะได้ 0
strlen(str)	หาความยาวข้อความ
strlwr(str)	แปลงข้อความภาษาอังกฤษใน str เป็นตัวพิมพ์เล็ก
strupr(str)	แปลงข้อความภาษาอังกฤษใน str เป็นตัวพิมพ์ใหญ่

ตัวอย่างการใช้ฟังก์ชัน strcpy และ strcat

```
1   #include <stdio.h>
2   #include <string.h>
3   int main(void){
4      char str1[30] = "My Birthday ";
5      char str2[30] = "18 August 1991";
6      char str3[30];
7      strcpy(str3, str1);
8      strcat(str1, str2);
9      printf("str1 = [%s]\n", str1);
10     printf("str3 = [%s]\n", str3);
11     return(0);
12  }
```

ผลการทำงาน

```
str1 = [My Birthday 18 August 1991]
str3 = [My Birthday ]
```

ฟังก์ชันสำหรับอักขระ ใน ctype.h

Function	Description
tolower(ch)	เปลี่ยนอักขระเป็นตัวพิมพ์เล็ก
toupper(ch)	เปลี่ยนอักขระเป็นตัวพิมพ์ใหญ่

ตัวอย่างการใช้ฟังก์ชัน tolower และ toupper

```
1   #include <stdio.h>
2   #include <ctype.h>
3   int main(){
4     int i;
5     char string[] = "This Is a String";
6     i=0;
7     while(string[i] != '\0'){
8       string[i] = toupper(string[i]);
9       i++;
10    }
11    printf("%s\n",string);
12    i=0;
13    while(string[i] != '\0'){
14      string[i] = tolower(string[i]);
15      i++;
16    }
17    printf("%s\n",string);
18    return(0);
19  }
```

ผลการทำงาน

```
THIS IS A STRING
this is a string
```

Outline ของฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้นเอง

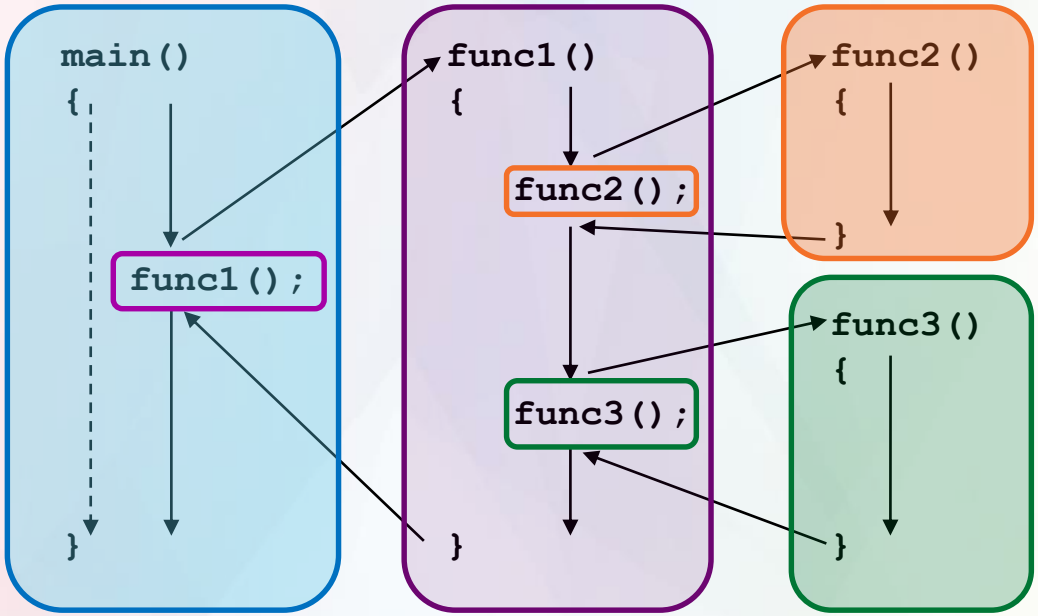
- ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้น
- โครงสร้างการเรียกใช้ Function
- โครงสร้างของฟังก์ชัน (Structure of Function)
- รูปแบบการรับ-ส่งค่าฟังก์ชัน
- Pass by Value
- Pass by Reference
- Function Prototype
- Local and Global Variables
- การส่งอาร์เรย์เป็นพารามิเตอร์เข้าสู่ฟังก์ชัน

ฟังก์ชันที่ผู้เขียนโปรแกรมสร้างขึ้น

ฟังก์ชันมาตรฐานที่ภาษาซีเตรียมไว้ให้แล้ว ในบางครั้งอาจไม่ตรงกับความต้องการของผู้เขียนโปรแกรม ภาษาซีจึงเปิดช่องทางให้ผู้เขียนโปรแกรมสามารถสร้างฟังก์ชันขึ้นเองได้ตามต้องการ เราเรียกฟังก์ชันลักษณะนี้ว่า **User-Defined Function**

โครงสร้างการเรียกใช้ฟังก์ชัน

ในการเขียนโปรแกรมด้วยภาษาซีต้องมีฟังก์ชัน main( ) เสมอ และหากต้องการให้โปรแกรมทำงานฟังก์ชันใด ฟังก์ชัน main( ) จะเป็นผู้เรียกฟังก์ชันนั้นขึ้นมาทำงาน ซึ่งในฟังก์ชันนั้นเองก็ยังสามารถเรียกใช้ฟังก์ชันอื่น ๆ ขึ้นมาทำงานได้อีกด้วย ดังตัวอย่าง



โครงสร้างของฟังก์ชัน

โครงสร้างของฟังก์ชัน มีรูปแบบดังนี้

```
return_type function-name(type para-1, type para-2, ... , type para-n)
{
   local variable-declarations ;
   statement-1 ;
   statement-2 ;
   ...
   statement-n ;
   return(value) ;
}
```

- return_type เป็นชนิดของผลลัพธ์ที่ได้จากการทำงานของฟังก์ชัน ในกรณีที่ไม่ต้องการส่งค่ากลับให้ใส่ void หากไม่ใส่จะมีค่าเท่ากับ int
- function-name คือ ชื่อของฟังก์ชันที่สร้างขึ้น ควรตั้งให้สื่อความหมาย และมีหลักการตั้งชื่อเช่นเดียวกับตัวแปร

โครงสร้างของฟังก์ชัน

โครงสร้างของฟังก์ชัน มีรูปแบบดังนี้

```
return_type function-name(type para-1, type para-2, ... , type para-n)
{
   local variable-declarations ;
   statement-1 ;
   statement-2 ;
   ...
   statement-n ;
   return(value) ;
}
```

type para-1, type para-2, ... , type para-n คือ formal parameter(s)

- เป็นตัวแปรที่ใช้รับข้อมูลเพื่อนำมาประมวลผลในฟังก์ชัน โดยต้องกำหนดชนิดของตัวแปรไว้ข้างหน้าตัวแปร
- หากมีตัวแปรมากกว่าหนึ่ง ให้คั่นแต่ละตัวด้วยเครื่องหมายจุลภาค แต่หากไม่มีการรับตัวแปรใดเลย ให้ใส่ void

โครงสร้างของฟังก์ชัน

โครงสร้างของฟังก์ชัน มีรูปแบบดังนี้

```
return_type function-name(type para-1, type para-2, ... , type para-n)
{
    local variable-declarations ;
    statement-1 ;
    statement-2 ;
    ...
    statement-n ;
    return(value) ;
}
```

- ชุดคำสั่งทั้งหมด จะต้องรวมอยู่ในวงเล็บปีกกา { }
- local variable-declarations เป็นการสร้างตัวแปรเพื่อใช้งานภายในฟังก์ชัน
- statement-1, statement-2, ... , statement-n ชุดคำสั่งที่รวมอยู่ภายในฟังก์ชัน

โครงสร้างของฟังก์ชัน

โครงสร้างของฟังก์ชัน มีรูปแบบดังนี้

```
return_type function-name(type para-1, type para-2, ... , type para-n)
{
   local variable-declarations ;
   statement-1 ;
   statement-2 ;
   ...
   statement-n ;
   return(value) ;
}
```

return(value) เป็นคำสั่งที่ใช้สำหรับคืนค่าออกจากฟังก์ชัน โดยชนิดของค่าที่คืนออกไปต้องเป็นชนิดเดียวกันกับ return_type ที่ประกาศไว้หน้าชื่อของฟังก์ชัน หากฟังก์ชันไม่มีการคืนค่า ไม่ต้องใช้คำสั่งนี้ และ return_type ต้องถูกกำหนดให้เป็น void

ตัวอย่างโปรแกรมที่ยังไม่ได้สร้างฟังก์ชัน

โปรแกรม

```
1  #include <stdio.h>
2  int main()
3  {
4    printf("********************\n");
5    printf("StarLine Printing\n");
6    printf("********************\n");
7    return(0);
8  }
```

ผลการทำงาน

```
********************
StarLine Printing
********************
```

ตัวอย่างการสร้างและเรียกใช้ฟังก์ชัน

โปรแกรม

```
 1  #include <stdio.h>
 2
 3
 4
 5
 6
 7  int main()
 8  {
 9    printf("********************\n");
10    printf("StarLine Printing\n");
11    printf("********************\n");
12    return(0);
13  }
```



แทรกช่องว่าง

ผลการทำงาน

```
********************
StarLine Printing
********************
```

ตัวอย่างการสร้างและเรียกใช้ฟังก์ชัน (ต่อ)

โปรแกรม

```
1  #include <stdio.h>
2
3       StarLine(     )
4
5
6
7  int main()
8  {
9    StarLine();
10   printf("StarLine Printing\n");
11   StarLine();
12   return(0);
13 }
```

ตั้งชื่อฟังก์ชันขึ้นมาเอง

บรรจุคำสั่งเข้าไปใน function

ใส่ชื่อฟังก์ชันเพื่อเรียกใช้งาน

ใส่ชื่อฟังก์ชันเพื่อเรียกใช้งาน

ตัวอย่าง การเรียกใช้ฟังก์ชันของโปรแกรม

โปรแกรม

```
1  #include <stdio.h>
2
3  void StarLine(void) // ไม่มีการรับ-ส่งค่า จึงใส่ void ทั้งสองตำแหน่ง
4  {
5    printf("**********\n");
6  }
7  int main()
8  {
9    StarLine();                  // เรียกฟังก์ชัน StarLine (โปรแกรมจะกระโดดเข้าไปทำงานใน StarLine)
10   printf("StarLine Printing\n");
11   StarLine();
12   return(0);
13 }
```

กำหนดขอบเขตฟังก์ชันด้วย { }

ผลการทำงาน

```
**********
StarLine Printing
**********
```

รูปแบบการรับ-ส่งค่าฟังก์ชัน

ฟังก์ชันสามารถรับ-ส่งค่าได้ 4 รูปแบบดังนี้

1. ไม่มีการรับ-ส่งค่าใด ๆ

ตัวอย่างเช่น `void function_name (void)`

หมายเหตุ: ฟังก์ชัน StarLine ที่ผ่านมา ไม่มีการรับ-ส่งค่า (void) StarLine (void)

2. รับค่าเข้าอย่างเดียว

ตัวอย่างเช่น `void function_name (int n)`

3. ส่งค่าออกอย่างเดียว

ตัวอย่างเช่น `float function_name (void)`

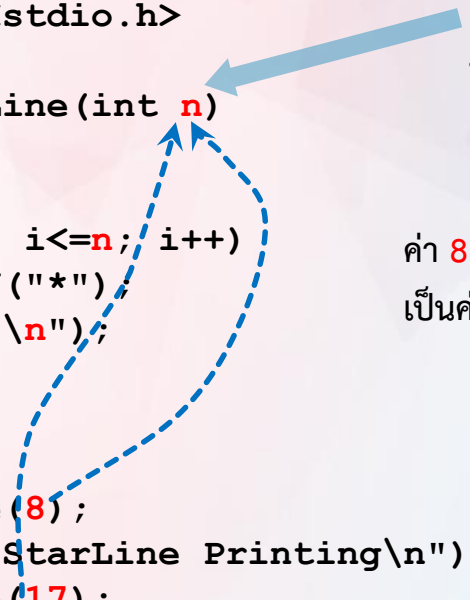
4. รับและส่งค่า

ตัวอย่างเช่น `float function_name (int n)`

ตัวอย่าง ฟังก์ชันที่มีการรับค่าเข้าอย่างเดียว

โปรแกรม

```
1  #include <stdio.h>
2
3  void StarLine(int n)
4  {
5    int i;
6    for(i=1; i<=n; i++)
7      printf("*");
8    printf("\n");
9  }
10 int main()
11 {
12   StarLine(8);
13   printf("StarLine Printing\n");
14   StarLine(17);
15   return(0);
16 }
```



n เป็น Formal Parameter เป็น local variable
ใช้เพื่อรับค่าที่ส่งเข้ามาในฟังก์ชันจาก Actual Parameter

ค่า 8 และ 17 เป็น Actual Parameter
เป็นค่าที่ต้องการส่งเข้าไปทำงานใน function

ผลการทำงาน

```
********
StarLine Printing
*****************
```

ตัวอย่าง ฟังก์ชันที่มีการส่งค่าออกอย่างเดียว

โปรแกรม

```
1  #include <stdio.h>
2  int get100(void) {
3    int num, flag = 0;
4    do {
5      printf("Please enter score (0-100): ");
6      scanf("%d",&num);
7      if(num >= 0 && num <= 100)
8        flag = 1;
9    } while(flag == 0);
10   return num;
11 }
12 int main(void) {
13   int score;
14   score = get100();
15   printf("The score entered is %d",score);
16   return 0;
17 }
```

เข้าไปทำงานใน get100 โดยไม่ส่งค่าเข้าไป

get100 คืนค่า num กลับมา ซึ่งเป็นชนิด integer ตามที่ระบุไว้ข้างหน้า fucntion

นำค่าที่ได้รับกลับมา ไปกำหนดให้กับ score

ผลการทำงาน

```
Please enter score (0-100):-5
Please enter score (0-100):99
The score entered is 99
```

ตัวอย่าง ฟังก์ชันที่มีการรับและส่งค่ากลับ

โปรแกรม

```
1  #include <stdio.h>
2  float FindArea(float radius)
3  {
4    return(3.14*radius*radius);
5  }
6  int main(void)
7  {
8    float radius, area;
9    printf("Enter Radius: ");
10   scanf("%f", &radius);
11   area = FindArea(radius);
12   printf("Area is %0.2f\n", area);
13   return(0);
14 }
```

FindArea คืนค่า ที่คำนวณได้กลับมา

เข้าไปทำงานใน FindArea โดยส่งค่า Actual Parameter ไปเก็บไว้ใน Formal Parameter (สามารถตั้งชื่อ Actual Parameter เหมือนกับ Formal Parameter ได้ เพราะคนละฟังก์ชันกัน)

นำค่าที่ได้รับกลับมา ไปกำหนดให้กับ area

ผลการทำงาน

```
Enter Radius: 5
Area is 78.50
```

การส่งข้อมูลเข้าสู่ฟังก์ชัน

ฟังก์ชันที่มีการรับค่า มี 2 แบบ ตามลักษณะของพารามิเตอร์ที่ส่งให้ฟังก์ชัน

1. Pass by Value

- เป็นการสำเนาค่าของ Actual parameter ให้ Formal Parameter
- หากค่าของ Formal Parameter เปลี่ยนไป จะไม่มีผลกับค่าของ Actual Parameter

2. Pass by Reference

- เป็นการส่งตำแหน่งหน่วยความจำของ Actual parameter ให้กับ Formal Parameter ทำให้ Formal Parameter กับ Actual Parameter เสมือนเป็นตัวแปรเดียวกัน
- หากค่าของ Formal Parameter เปลี่ยนไป ค่าของ Actual Parameter จะเปลี่ยนด้วย

ตัวอย่าง Pass by Value

```
 1  #include <stdio.h>
 2  float FindArea(float radius)
 3  {
 4    return(3.14*radius*radius);
 5  }
 6  int main(void)
 7  {
 8    float radius, area;
 9    printf("Enter Radius: ");
10    scanf("%f", &radius);
11    area = FindArea(radius);
12    printf("Area is %0.2f\n", area);
13    return(0);
14  }
```

เข้าไปทำงานใน FindArea
โดยส่งค่า Actual Parameter
ไปเก็บไว้ใน Formal Parameter

FindArea คืนค่า ที่คำนวณได้กลับมา

นำค่าที่ได้รับกลับมา ไปกำหนดให้กับ area

ผลการทำงาน

```
Enter Radius: 5
Area is 78.50
```

Pass by Reference

Pass by Reference เป็นการส่งตำแหน่งหน่วยความจำของ Actual Parameter ให้กับ Formal Parameter ทำให้ Formal Parameter เสมือนเป็นตัวแปรเดียวกันกับ Actual Parameter

องค์ความรู้พื้นฐานที่จำเป็นต่อการทำความเข้าใจ Pass by Reference

- ตำแหน่งของหน่วยความจำ (Memory Address)
- ตัวแปร Pointer

ตำแหน่งหน่วยความจำ (Memory Address)

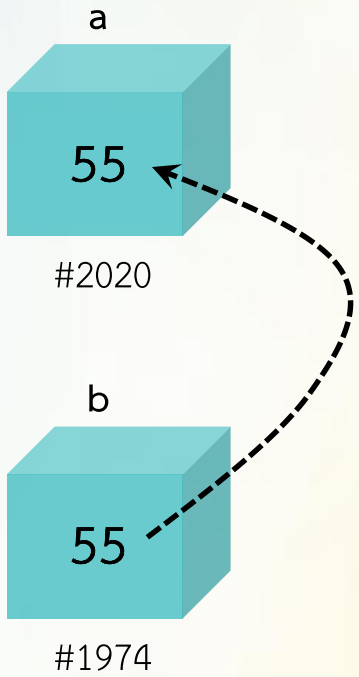
ตัวแปรแต่ละตัวจะถูกสร้างขึ้นใน RAM และมีหมายเลขตำแหน่งใน RAM เรียกว่า “Address” เมื่อเรากำหนดค่าให้กับตัวแปร ค่านั้นจะถูกเก็บไว้ใน RAM ที่ Address ของตัวแปรนั้น ดังตัวอย่าง

```
int a;
```
สร้างกล่องชื่อว่า a เพื่อเก็บข้อมูลชนิด integer และกล่องนี้อยู่ ณ Address #2020

```
a = 19;
```
นำค่า 19 ไปเก็บไว้ในกล่องชื่อว่า a ซึ่งก็คือ การนำค่า 19 ไปเก็บไว้ใน Address #2020 ของ RAM

```
int b = 55;
```
สร้างกล่องชื่อว่า b เพื่อเก็บข้อมูลชนิด integer และกล่องนี้อยู่ ณ Address #1974

```
a = b;
```
สำเนาค่าในกล่อง b ไปเก็บไว้ในกล่อง a ซึ่งก็คือ การสำเนาค่าใน RAM Address #1974 ไปเก็บไว้ใน RAM Address #2020



Pass by Reference (ต่อ)

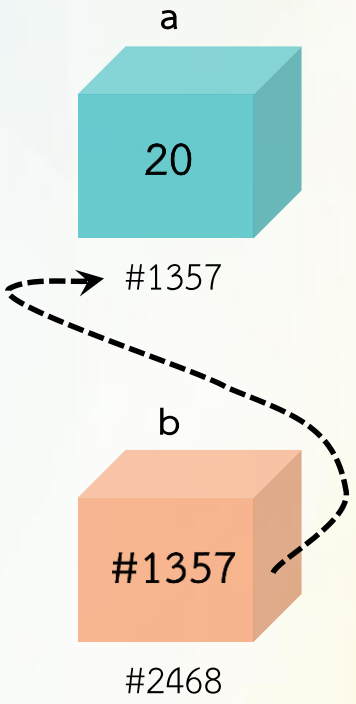
ตัวแปร Pointer

เป็นตัวแปรที่ใช้เก็บ Address (ตำแหน่งในหน่วยความจำ) ของตัวแปร ซึ่งแตกต่างกับตัวแปรชนิดพื้นฐานที่สร้างขึ้นมาเพื่อเก็บข้อมูล เช่น จำนวนเต็ม จำนวนจริง ฯลฯ

```
int a = 20;
```
สร้างตัวแปร a เพื่อเก็บข้อมูล integer พร้อมกับกำหนดค่า 20 เข้าไป และตัวแปรนี้อยู่ใน RAM Address #1357

```
int *b;
```
เป็นการสร้าง pointer ชื่อว่า b อยู่ใน Address #2468

```
b = &a;
```
กำหนดค่า Address ของ a (#1357) ให้กับ pointer b

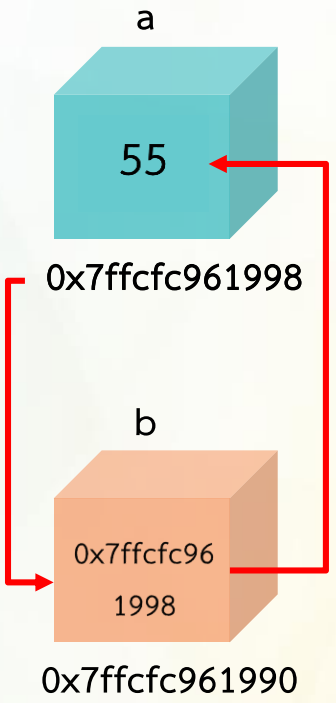


ดังนั้น เมื่อเข้าถึงข้อมูลใน b จะพบ Address ของ a (#1357) และด้วย address นี้เอง ทำให้เราสามารถเข้าถึงข้อมูลใน a (20) ได้

ตัวอย่าง Pass by Reference

```
1  #include <stdio.h>
2  int main(void) {
3    int a=20;
4    int *b;    //การสร้างตัวแปร pointer ชื่อ b
5    printf("a ถูกจองอยู่ใน RAM Address ที่ %p และเก็บค่า %d ไว้แล้ว\n",&a,a);
6    b = &a;
7    printf("pointer b ถูกจองใน RAM Address ที่ %p และเก็บค่า %p\n",&b,b);
8    printf("ซึ่งเป็น Address ของ a\n");
9    *b = 55;   // *b หมายถึง ข้อมูลของตัวแปร ณ Address ที่ b เก็บไว้
10   printf("ข้อมูลในตัวแปร a ถูกเปลี่ยนเป็น %d (ผ่านทาง pointer b)\n",a);
11   return 0;
12 }
```

%p คือ format string ที่ใช้แสดงค่า Address ของตัวแปร



ผลการทำงาน

a ถูกจองอยู่ใน RAM Address ที่ 0x7ffcfc961998 และเก็บค่า 20 ไว้แล้ว

pointer b ถูกจองใน RAM Address ที่ 0x7ffcfc961990 และเก็บค่า 0x7ffcfc961998

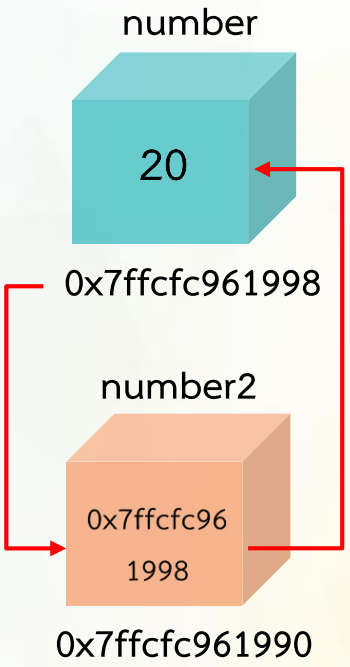
ซึ่งเป็น Address ของ a

ข้อมูลในตัวแปร a ถูกเปลี่ยนเป็น 55 (ผ่านทาง pointer b)

- &ชื่อตัวแปร = Address ของตัวแปร
- var-type *var-name = การสร้างตัวแปรชนิด pointer
- ชื่อ pointer = Address ที่เก็บอยู่ใน pointer
- *ชื่อ pointer = ค่าของตัวแปร ณ Address ที่เก็บไว้ใน pointer

ตัวอย่าง Pass by Reference

```
 1  #include <stdio.h>
 2  void NumChange(int *number2){
 3    printf("Before change : %d\n",*number2);
 4    *number2 += 5;
 5    printf("After change  : %d\n",*number2);
 6  }
 7  int main(void){
 8    int number = 15;
 9    NumChange(&number);
10    printf("After entering the function : %d\n",number);
11    return(0);
12  }
```



ผลการทำงาน

```
Before change : 15
After change  : 20
After entering the function : 20
```

การทำ Pass by Reference
ไม่สามารถให้ Actual Parameter เป็นค่าคงที่ได้
เพราะค่าคงที่ไม่มีการจองหน่วยความจำ
จึงไม่มี Address ส่งให้ Formal Parameter
ซึ่งเป็นตัวแปรชนิด Pointer

Function Prototype

ฟังก์ชันสามารถมองเห็นและเรียกใช้งานฟังก์ชันที่อยู่บรรทัดบนกว่าได้ เนื่องจากฟังก์ชันส่วนบนนั้นได้ผ่านการแปลโดย Compiler เรียบร้อยแล้ว

```
#include <stdio.h>

void StarLine(void)
{...}
void NumChange(int number)
{...}
float FindArea(float radius)
{...}
int main()
{...}
```

ยังไม่รู้จัก NumChange and FindArea จึงเรียกใช้สองฟังก์ชันนี้ไม่ได้

รู้จัก StarLine แล้วจึงเรียกใช้งานได้ แต่ยังไม่รู้จัก FindArea

รู้จัก StarLine และ NumChange แล้วจึงเรียกใช้สองฟังก์ชันนี้ได้

รู้จักทั้ง 3 ฟังก์ชันแล้วจึงเรียกใช้ทั้ง 3 ฟังก์ชันได้

หากฟังก์ชันที่อยู่ด้านบนต้องการเรียกฟังก์ชันที่อยู่ด้านล่าง จำเป็นต้องประกาศ Prototype ของฟังก์ชันทั้งหมดไว้ก่อน

Function Prototype (ต่อ)

Prototype เปรียบเสมือนโครงสร้างโดยคร่าวของ Function
ซึ่งประกอบไปด้วย var_type, Function Name และ Parameter Set

ตำแหน่งของ Prototype ควรอยู่บริเวณต้นโปรแกรม (ถัดจาก Preprocessor Directives) หรือบริเวณต้นของฟังก์ชัน main

- เพื่อที่ Compiler จะได้รู้จักชื่อและโครงสร้าง (return_type and parameter) ของฟังก์ชัน
- และสามารถแปลคำสั่งที่มีการเรียกใช้ฟังก์ชันที่สร้างไว้ในบรรทัดล่างกว่าได้ (เพราะได้รู้จักโครงสร้างโดยคร่าวที่ประกาศไว้ก่อนหน้านี้แล้ว)

Function Prototype (ต่อ)

ตัวอย่างโครงสร้างโปรแกรมที่มีการสร้าง Prototype

```
#include <stdio.h>

void StarLine(void);
void NumChange(int number);
float FindArea(float radius);

int main(void)
{...}
void StarLine(void)
{...}
void NumChange(int number)
{...}
float FindArea(float radius)
{...}
```

Function Prototypes

Function

ฟังก์ชัน StarLine สามารถเรียกใช้ฟังก์ชัน NumChange และ FindArea ได้ เนื่องจาก Compiler รู้จักชื่อ, ข้อมูลเข้า (Formal Parameter), และข้อมูลออก (Return Type) ของฟังก์ชัน ตั้งแต่ตอนสร้าง Prototype ที่ต้นโปรแกรม

Function Prototype (ต่อ)

```
1  #include <stdio.h>
2  void StarLine(void);                  // Prototype of StarLine Function
3  int main()
4  {
5    StarLine();                         // Function Call
6    printf("StarLine Printing\n");
7    StarLine();                         // Function Call
8    return(0);
9  }
10 void StarLine(void)                   // StarLine Function
11 {
12   printf("**********************\n");
13 }
```

ผลการทำงาน

```
**********************
StarLine Printing
**********************
```

Function Prototype (ต่อ)

```
1  #include <stdio.h>
2  int ADD_AB(int A, int B);          // Function Prototype
3  int main()
4  {
5    int A=2, B=7, C;
6    C = ADD_AB(A,B);                 // Function Call
7    printf("Answer is %d",C);
8    return(0);
9  }
10 int ADD_AB(int A, int B)           // Function
11 {
12   int x;
13   x = A+B;
14   return(x);
15 }
```

ผลการทำงาน

```
Answer is 9
```

ชนิดของตัวแปรแยกตามขอบเขตการทำงาน

- **ตัวแปรชนิดโกลบอล (Global Variable)** เป็นตัวแปรที่ประกาศไว้นอกฟังก์ชัน แนะนำให้ประกาศไว้ตอนต้นของโปรแกรม จะได้เรียกใช้งานได้ทุกที่ และทุกฟังก์ชันในโปรแกรม
- **ตัวแปรชนิดโลคอล (Local Variable)** เป็นตัวแปรที่ประกาศไว้ในฟังก์ชัน และมีผลแค่ภายในฟังก์ชันเท่านั้น (ตัวแปรจะถูกทำลายเมื่อออกจากฟังก์ชัน) หากมีชื่อซ้ำกับ Global variable จะถือว่าเป็นคนละตัวแปรกัน

ชนิดของตัวแปรแยกตามขอบเขตการทำงาน (ต่อ)

Global variable

```
1   #include <stdio.h>
2   int num = 19;
3   void increase( ){
4     num = num + 5;
5   }
6   void main( ){
7     printf("num = %d\n", num);
8     increase();
9     printf("num = %d\n", num);
10  }
```

19+5 = 24

ผลลัพธ์

```
num = 19

num = 24
```

Local variable

```
1   #include <stdio.h>
2   void increase( ){
3     int num = 2;
4     num = num + 5;
5   }
6   void main( ){
7     int num = 19;
8     printf("num = %d\n", num);
9     increase();
10    printf("num = %d\n", num);
11  }
```

2+5 = 7

ผลลัพธ์

```
num = 19

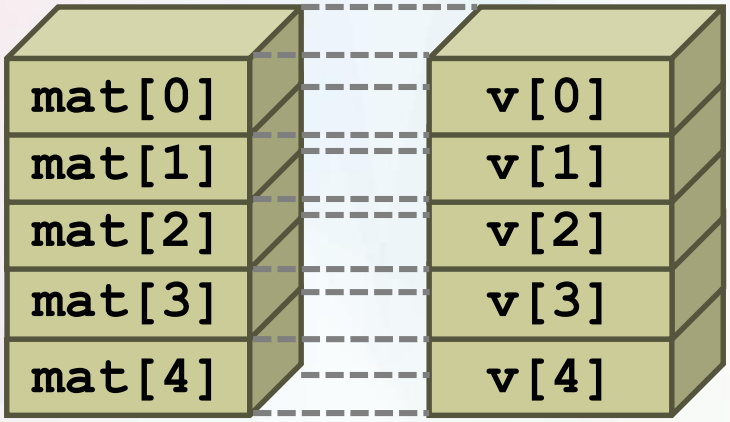
num = 19
```


Computer Engineering • Faculty of Engineering • Rajamangala University of Technology Thanyaburi

การส่งอาร์เรย์เป็นพารามิเตอร์ให้กับฟังก์ชัน

เป็นการส่งข้อมูลเข้าฟังก์ชันแบบ Pass by Reference เนื่องจาก

- ไม่ต้องจองหน่วยความจำอีก 1 ชุดที่มีขนาดเท่ากับ Array เพื่อรับข้อมูลจาก Actual Parameter ทำให้ประหยัดพื้นที่หน่วยความจำ
- ลดการประมวลผล ในการสำเนาข้อมูลจาก Actual สู่ Formal Parameter
- ลดการประมวลผล ในการส่งค่า Array ทั้งชุดออกจากฟังก์ชัน (กรณีที่ต้องการส่ง Array กลับมาประมวลผลต่อ)



ตัวอย่าง ฟังก์ชันที่รับค่าเป็นอาร์เรย์

```
1  #include <stdio.h>
2  void Dollar(float baht[], int num)
3  {
4    int i;
5    printf("Before changing : %.2f\n", baht[0]);
6    for(i=0; i<num; i++)
7      baht[i] *= 40.0;
8    printf("After changing  : %.2f\n", baht[0]);
9  }
10 int main()
11 {
12   float money[3] = {200.0, 400.0, 600.0};
13   Dollar(money, 3);
14   printf("After entering function: %.2f\n", money[0]);
15   return(0);
16 }
```

- เหมือนการประกาศ Array
- สามารถไม่ระบุขนาดได้ เฉพาะมิติแรกเท่านั้น

อ้างแค่ชื่อ Array ในตำแหน่ง Actual Parameter พร้อมส่งขนาดของ Array

ผลการทำงาน

```
Before changing : 200.00
After changing  : 8000.00
After entering function: 8000.00
```

ตัวอย่าง ฟังก์ชันที่รับค่าเป็นอาร์เรย์

```
1   #include <stdio.h>
2   int max_of(int v[],int n);
3   int main(){
4     int mat[5] = {45,36,72,90,100};
5     int max_m;
6     max_m = max_of(mat,5);
7     printf("max value of score %d",max_m);
8
9   }
10  int max_of(int v[],int n) {
11    int max = v[0];
12    for (int i = 1; i< n; i++){
13      if(v[i]> max)
14         max = v[i];}
15    return max;
16  }
```

ผลการทำงาน

```
max value of score 100
```


Computer Engineering ● Faculty of Engineering ● Rajamangala University of Technology Thanyaburi

ตัวอย่าง ฟังก์ชันที่รับค่าเป็นอาร์เรย์

```
 1  #include <stdio.h>
 2  void printArray(int [][3]);
 3  int main(void) {
 4    int array1[2][3] = {{1,2,3},{4,5,6}},
 5        array2[2][3] = {1,2,3,4,5},
 6        array3[2][3] = {{1,2},{4}};
 7    printf("Value in array1 by row are:\n");
 8    printArray(array1);
 9    printf("Value in array2 by row are:\n");
10    printArray(array2);
11    printf("Value in array3 by row are:\n");
12    printArray(array3);
13    return 0;
14  }
15  void printArray(int a[][3]) {
16    int i,j;
17    for (i=0; i<2; i++) {
18      for (j=0; j<3; j++)
19        printf("%d ",a[i][j]);
20      printf("\n");
21    }
22  }
```

ผลการทำงาน

```
Value in array1 by row are:
1 2 3
4 5 6
Value in array2 by row are:
1 2 3
4 5 0
Value in array3 by row are:
1 2 0
4 0 0
```

ตัวอย่าง การส่งผ่านค่าสตริงระหว่างฟังก์ชัน

```
1  #include <stdio.h>
2  void greeting(char []);
3  void main (void){
4      char string[20];
5      printf("What is your name => ");
6      gets(string);
7      greeting(string);
8  }
9  void greeting(char name[]){
10     printf("Hello there %s!" , name );
11 }
```

มีหลักการเหมือนการส่ง Array เข้าสู่ฟังก์ชัน
เนื่องจาก String เป็น Array of Character

ผลการทำงาน

```
What is your name => Harry Potter
Hello there Harry Potter!
```

ตัวอย่าง การส่งผ่านค่าสตริงระหว่างฟังก์ชัน

```
1  #include <stdio.h>
2  void greeting(char [] );
3  void main (void)
4  {    char string[20];
5       greeting(string);
6       printf("You are scaring me: %s!\n" , string);
7  }
8  void greeting(char name[]){
9       printf("Enter your first name => ");
10      gets(name);
11 }
```

ข้อมูลใน string เปลี่ยนตาม name ในฟังก์ชัน greeting

เปลี่ยนแปลงข้อมูลในฟังก์ชัน greeting โดยการรับข้อมูลจากฟังก์ชัน gets

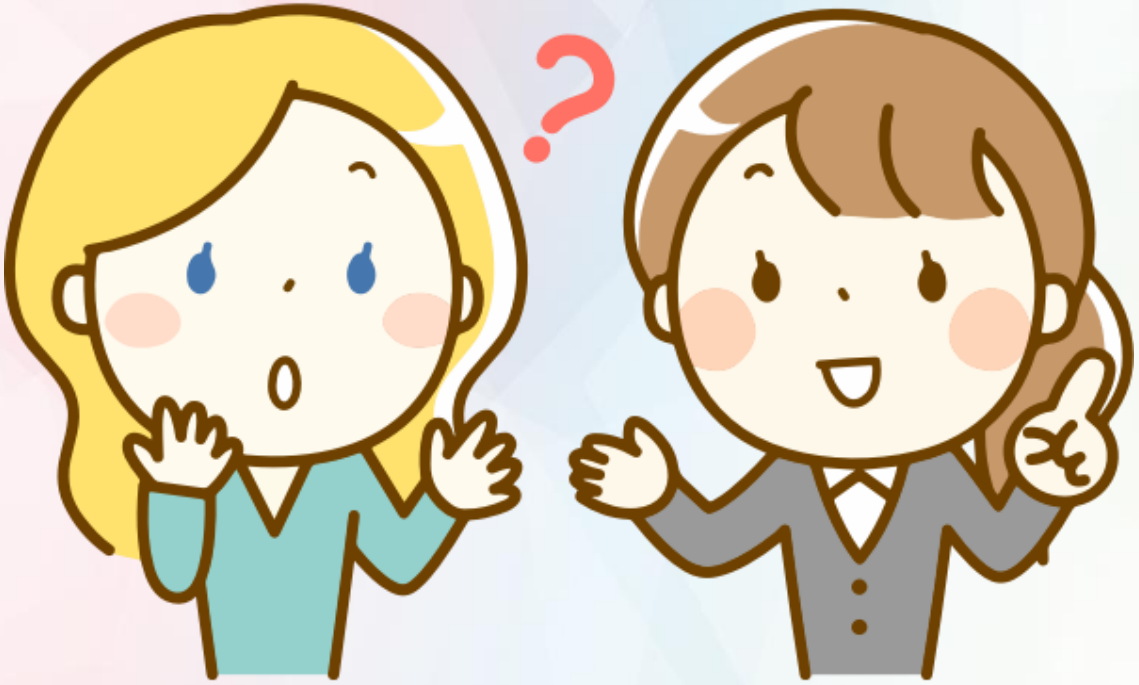
ผลการทำงาน

```
Enter your first name => Voldemort
You are scaring me: Voldemort!
```

สรุป การส่งผ่านค่าอาร์เรย์และสตริงระหว่างฟังก์ชัน

Formal Parameter ที่รับค่าอาร์เรย์หรือสตริง สามารถไม่ระบุขนาดของอาร์เรย์มิติแรก แต่มิติที่เหลือจำเป็นต้องระบุ

```
void Dollar(float baht[], int num) {...}
void modifyArray(int b[], int size) {...}
void printArray(int a[][3]) {...}
void greeting(char name[]) {...}
```



Questions & Answers